Concurrency in Haskell

Haskell's *MVar* is a key building block for concurrency As we shall see, it allows a mixture of: shared memory concurrency and message passing concurrency.

```
import Control.Concurrent
forkIO :: IO () -> IO ThreadId
```

Kicks off the computation asynchronously in a new thread; immediately returns the new thread's id. forkIO causes a thread to run asynchronously. When main ends, the program is terminated even if threads are still running.

MVar a - mutable variable storing a value of type a

An MVar is either empty (holds no value) or full (holds a value of type a).

Create a new MVar with a given value:

```
newMVar :: a -> IO (MVar a)
```

Create a new empty MVar:

```
newEmptyMVar :: IO (MVar a)
```

Take value from an MVar:

```
takeMVar :: MVar a -> IO a
```

Blocks until the MVar is full, leaves the MVar empty. Yields the value that was in the MVar.

Put value into an MVar:

```
putMVar :: MVar a -> a -> IO ()
```

Blocks until the MVar is empty. Leaves the MVar full, containing the given value.

Read current value from MVar:

```
readMVar :: MVar a -> IO a
```

Blocks until the MVar is full. Yields the current value of the MVar, leaving it full.